

Session 3 Homework — Recently Viewed, on Redux

CORE, STRETCH, and CHALLENGE are all required — every one of them must be completed. The three names describe increasing difficulty only, nothing about which parts you submit.

Context

This extends the same B4F Hub project. It is **not** a separate project — it continues from where Session 3 ended.

In class we moved **saved opportunities** out of Context and into a Redux Toolkit slice. **Recently viewed** is still a Context. Your job is to move it too, using exactly the same pattern, and then go a little further.

This is not because Context was the wrong choice for recently viewed — it worked fine there through all of class. It is a deliberately chosen second practice feature, so that you repeat the Context-to-Redux pattern yourself, on a different concern, and can compare the same feature implemented both ways. In a real application, keeping a small concern like this in Context could still be a perfectly valid, permanent choice.

`NotificationContext` stays as Context — permanently, not as a matter of practice. Do not touch it.

Goal

Manage the recently-viewed list in the Redux store, with the same behaviour it has today, plus the ability to remove a single entry.

What to build

A `recentlyViewedSlice` registered in the existing store, replacing `RecentlyViewedContext` completely.

CORE (required)

1. A new file `client/src/store/recentlyViewedSlice.ts` holding:
 - the state shape — an object with an `entries` array of `{ id, title }`;
 - a `recordView` reducer that keeps the current behaviour exactly: no duplicates, newest first, at most five entries;
 - a `clearRecentlyViewed` reducer that empties the list.
2. Register the new reducer in `client/src/store/store.ts` under the key `recentlyViewed`. Do not create a second store.
3. `OpportunityDetailPage` dispatches `recordView` instead of calling the Context function — still only when a matching opportunity was actually found.
4. `RecentlyViewedList` reads the entries with `useSelector` and dispatches `clearRecentlyViewed` from its Clear button.

5. Delete `client/src/context/RecentlyViewedContext.tsx` and remove its Provider from `main.tsx`.
Do this **last** — after every consumer has moved.

When CORE is done the app must behave exactly as it did before. Nobody using the app should be able to tell that anything changed.

STRETCH (required)

1. Add two named selector functions to the slice file:
 - `selectRecentlyViewed(state)` — the whole list;
 - `selectRecentlyViewedExcluding(state, excludeId?)` — the list without one entry.
2. Change `RecentlyViewedList` so it selects exactly what it renders, using the second selector instead of selecting everything and filtering inside the component.

After this step, no component should contain the string `state.recentlyViewed.entries`.

CHALLENGE (required)

1. Add a `removeRecentlyViewed` reducer that takes an opportunity id and removes only that entry.
2. Export its generated action creator.
3. Add a `selectRecentlyViewedCount(state)` selector, **and use it** — update the Clear button so it reads `Clear (3)` instead of a bare `Clear`. A selector with no consumer is a leftover; give this one a real one.
4. Give every chip in the strip a small remove control (×) that dispatches `removeRecentlyViewed` with that entry's id. Give it an `aria-label` that names the opportunity — a bare × tells a screen-reader user nothing.
5. Style it so it sits beside the chip rather than wrapping onto its own line.

Clicking the remove control must **not** navigate to the opportunity.

Rules

Allowed — everything taught by the end of Session 3: `configureStore`, `createSlice`, `PayloadAction`, `<Provider store>`, `useSelector`, `useDispatch`, named selectors, `RootState`.

Not yet (do not use these) — `createAsyncThunk`, `extraReducers`, RTK Query, `createSelector`, custom middleware, any persistence library, `useReducer`. If you find yourself reaching for one of these, you have left the scope of this homework.

Do not change the server. Do not change `NotificationContext`.

Test it

- Visit three different opportunities: all three appear in the strip, newest first.
- Visit one of them again: it moves to the front, and is not duplicated.
- Visit six in total: only five are kept.

- On an opportunity's own detail page, that opportunity is not listed in its own strip.
- The Clear button reads `Clear (3)` after visiting three opportunities.
- Remove the middle entry: only it disappears, the others keep their order, and the Clear button's count drops to `Clear (2)`.
- Press Clear: the strip disappears entirely.
- Reload the page: the list is empty again. **This is correct** — neither Context nor Redux persists anything by itself.
- `npm run build` and `npm run lint` both pass with 0 errors.

Done when

- CORE, STRETCH and CHALLENGE are all implemented — all three are required.
- `client/src/context/RecentlyViewedContext.tsx` no longer exists.
- No component contains the string `state.recentlyViewed.entries`.
- `npm run build` and `npm run lint` pass with 0 errors.

Working time

Expect about **2 to 3 hours**. Due before Session 4.

Submitting

Commit and push to your own `student-<your-github-username>` branch, as usual. Do not push to `main`.